THE OFFICIAL FIELD MANUAL · VOLUME 1 · 2026

LD DEBLOAT GUIDE

How to strip **LDPlayer 9** down to a clean, fast, silent Android emulator — and keep it that way after every update and every launch.



EVERY STEP.
EVERY TRICK.
EVERY ERROR SOLVED.

WHAT YOU NEED	Only LDPlayer 9 on Windows. No programming skills required — every command is copy–pasteable; every failure has its fix listed.
RESULT	No LDPlayer ads · no pop-ups · no crash reporter · no app-store install redirects · no auto-start service · a tuned instance that survives every LDPlayer update by healing itself
TIME	~90 minutes once, then ~2 minutes per update forever
RISK	Nothing is deleted — every removed file is moved to a vault folder. Windows 10 (including Lite) is never modified: only two marked, reversible additions are ever made.

Contents

EVERY SECTION, IN ORDER · READ §1 FIRST

01 **Read me first**
What this is · what you need · result

02 **The bloat, itemized**
What LDPlayer ships and what it does

03 **The rules**
Eight rules that keep you out of trouble

04 **Know your target**
What to check before you change anything

05 **The Steps, 0–13**
The complete debloat · copy–pasteable

06 **Errors & fixes**
The complete field list

07 **Your new toolkit**
Every script this creates · when to run it

08 **Difficulty × time × risk**
The matrix at a glance

09 **Glossary**
Plain English, A–Z

010 **FAQ**
Direct answers

011 **Windows 10 Lite**
What is added (two things) and what is not

012 **Why it survives updates**
The permanence model

013 **Final checklist**
Tape it to the monitor

014 **Appendix: the scripts**
Every file reproduced verbatim

START HERE

Read me first 01

What this is

A precision debloat of LDPlayer 9. It removes, disables or walls off every component whose job is advertising at the user, reporting on the user, or selling promoted games to the user, while leaving every component responsible for running games untouched.

What you will end with

No pop-ups. No featured-games wall. No install interception. No crash reporter. No auto-start service at logon. A guest sized for stability. And a small enforcement layer that re-applies the debloat automatically after updates, because LDPlayer updates re-ship its own bloat silently.

What you need

- Windows with LDPlayer 9 installed (paths in this guide assume `C:\LDPlayer\LDPlayer9`).
- Administrator rights. Right-clicking and choosing "Run as administrator" is enough; the scripts here self-elevate.

- Roughly 90 minutes the first time. The hands-on work is ~15 minutes; the rest is understanding, so it never needs doing again.

What you will NOT get

This guide does not modify the Android image, game data, or Windows itself. It does not repack LDPlayer, does not bypass license checks, does not unlock paid features. It removes the advertising payload LDPlayer ships with because LDPlayer runs without it.

THE ENEMY

The bloat, itemized 02

LDPlayer 9 installs with three classes of non-essential software. Know them by shape, not by file name: names change between builds, these classes do not.

Class 1 — The advertising layer

WHAT IT IS	WHERE IT LIVES	WHAT IT DOES
ldplayerpartner.exe & ldplayerpartner2.exe	The install root	Pop-up ad launcher; fires on launch and on a timer
upload\ folder — **ldupload.exe** + OBS SDK runtime	`LDPlayer9\upload\`	Crash telemetry; ships data to a third-party storage bucket
launcherskin\ images + en.mp4	`LDPlayer9\launcherskin\`, `LDPlayer9\data\`	Ad creative shown inside the launcher
recommendConfigs\ + dir_*.dir	`LDPlayer9\vms\recommendConfigs\`	"Key-mapping for you" game suggestions with ad placements baked in
LDPlayerENSvr service	Windows services (Automatic)	Auto-starts the launcher plumbing, including the ad path, before you click anything

Class 2 — The interception layer

WHAT IT IS	WHERE IT LIVES	WHAT IT DOES
ld_launcher intercept feed (~460 KB JSON)	`Documents\XuanZhi9\Applications\ld_launcher\`	Maps Google Play packages to LD-store twins; on an intercepted install, redirects you to LD-store instead
cpi.data	`Documents\XuanZhi9\Applications\`	The ad-slot map: which screen gets which units (LauncherBottom, AppStoreSearch, AppStoreMain)
fetch stamp (hex name)	`Documents\XuanZhi9\Applications\`	Last-fetch date for the ad feed; tells LD when to pull again

Class 3 — The maintenance layer pretending to be plumbing

WHAT IT IS	WHERE IT LIVES	WHAT IT DOES
bugreport.exe (two copies)	The install root AND `ldmutiplayer\`	Crash reporter that also uploads; exists in both processes
dnrepairer.exe	The install root	"Repair" tool that re-downloads everything the debloat removed
LDPlayer_<build>.exe	The install root	Installer stub left behind by every update (~800 MB)
the partner report data	`%APPDATA%\XuanZhi9\partner_report.data`	Timestamp of last successful report POST (proves it reported)
the live ad cache	`%APPDATA%\XuanZhi9\cache\`	Holds re-downloaded ad images/configs; seal it with one ACL deny rule

i **Single most important fact:** LDPlayer re-downloads Class 2 and 3 on launch and on update, forcibly. Any cleanup that runs only once is already undone by the second thing you do. Every fix in this guide is engineered against exactly that.

READ TWICE

The rules 03

★ **RULE 1 — Rename first. Never delete.** Every removal in this guide is a move, not a delete. If something breaks, you can reason back. You cannot reason back from a deletion.

★ **RULE 2 — Read the logs, never guess.** VBox.log, LD's own log files and the console output tell you what actually happened. If a change disappears, the log says why.

★ **RULE 3 — Measure before you conclude.** "The debloat broke the emulator" is never a starting point. Look first (VBox.log, `pm list packages`, `verify-debloat.ps1`), conclude second.

★ **RULE 4 — Bisect with last-known-good.** Change one variable at a time and re-test. Fancier fixes fail for teeny reasons, not grand ones.

★ **RULE 5 — Verify before you destroy.** If a verification step prints something odd, stop and look at it by hand. Unknown output is a red flag, not a green light.

★ **RULE 6 — Never strip ACLs without setting replacements in the same move.** Removing permissions outright can make a folder unreachable even for you. Always deny exactly one thing, replace nothing else.

★ **RULE 7 — Pattern-match, never hardcode.** LD renames its own packages between builds; grep for patterns, never a fixed string.

★ **RULE 8 — Updates undo debloats. Plan for it.** Every fix in this guide assumes an update will one day restore the bloat on its own. The logon task and the Clean Launch watcher exist precisely because updates always do.

KNOW YOUR SHIP

Know your target 04

Two minutes before you touch anything. This tells you exactly what this box is running.

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\ldconsole.exe" list
& "C:\LDPlayer\LDPlayer9\ldconsole.exe" list2
(Get-Item "C:\LDPlayer\LDPlayer9\dnplayer.exe").VersionInfo.ProductVersion
Get-Service LDPlayerENSvr | Select-Object Status, StartType
Get-CimInstance Win32_OperatingSystem | Select-Object @{n="FreeRAM_GB";e=
{[math]::Round($_.FreePhysicalMemory/1MB,2)}}
Get-ChildItem "C:\LDPlayer\LDPlayer9" | Sort-Object Length -Descending | Select-Object -First
8 Name, Length
```

What you are looking for

- **The LDPlayer version** (file version of `dnplayer.exe`). Updates change what the bloat is called, never the classes above.
- **The instance count and state.** Under `C:\LDPlayer\LDPlayer9\vms\leidian0\` you will find one folder per instance.
- **LDPlayerENSvr** — usually Auto; you will set it to Disabled on Step 7.
- **Two config files** worth a look before editing anything: `vms\config\leidians.config` (update/telemetry pins) and `vms\config\leidian0.config` (instance CPU/RAM).
- **Shared folders** — where LD exposes host folders to the guest: `Documents\XuanZhi9\{Applications, Pictures, Misc}`.

i **Stop and screenshot** the launcher window (and the two config files) before you change anything. Not every problem goes away; but every problem becomes obvious after the first look.

THE MEAT

The Steps, 0–13 05

Difficulty: EASY · MEDIUM · HARD · EXPERT · Every command shown here works as printed. Where your install differs, change the path, not the shape of the step.

0 The 30-second symptom check

EASY ~1 min risk: none

WHY

Confirm that the thing that is bugging you is LDPlayer and not Windows, your keyboard driver, or your browser.

LOOK FOR

- Ads, recommendations or app-store suggestions inside the LDPlayer window you never asked for.
- Desktop pop-ups when LDPlayer is not running.
- Slow boots into the guest or a guest that freezes at roughly the same second on every cold boot.

IF YES

Proceed. If none of those apply, close this guide — you do not have the disease it treats.

1 Install the toolkit (2 minutes)

EASY ~2 min risk: none

DO THIS

Copy the five scripts shipped in this guide's repository (folder `scripts/`) into `C:\LDPlayer\`:

LD · COMMAND

```
New-Item -ItemType Directory -Force -Path C:\LDPlayer | Out-Null
Copy-Item ".\scripts\reapply-debloat.ps1"   C:\LDPlayer\reapply-debloat.ps1
Copy-Item ".\scripts\clean-launch.ps1"      C:\LDPlayer\clean-launch.ps1
Copy-Item ".\scripts\android-refresh.ps1"   C:\LDPlayer\android-refresh.ps1
Copy-Item ".\scripts\verify-debloat.ps1"    C:\LDPlayer\verify-debloat.ps1
Copy-Item ".\scripts\verify-android.ps1"    C:\LDPlayer\verify-android.ps1
```

Or copy the five files by hand with Explorer — drag & drop into `C:\LDPlayer`. That is the whole install.

TEST IT

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\verify-debloat.ps1
```

Expect a list of `[PASS]` lines and a final `RESULT` line. The first run may show FAILs for things the debloat fixes — that is expected; run it again after Step 3.

2 Photo the evidence BEFORE changing anything

EASY ~5 min risk: none

WHY

Enables you (or anyone helping you) to prove what was original after the fact, and forces you to look at the actual install before any checklist.

DO THIS

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\ldconsole.exe" list
& "C:\LDPlayer\LDPlayer9\ldconsole.exe" list2
(Get-Item "C:\LDPlayer\LDPlayer9\dnplayer.exe").VersionInfo.ProductVersion
Get-Service LDPlayerENSvr
Get-Content "C:\LDPlayer\LDPlayer9\vms\config\leidians.config" -ErrorAction
SilentlyContinue | Select-Object -Last 6
```

LDPlayer 9 installs at `C:\LDPlayer\LDPlayer9` in the default case. If yours lives elsewhere, substitute your install path; every step in this guide finds the folder itself.

3 Wave 1 — vault the classic bloat

EASY ~3 min risk: none — moves only, zero deletes

WHAT IT DOES

`reapply-debloat.ps1` moves the known offenders into a timestamped vault folder. It never deletes — if something breaks, the files are visible, named and in one place.

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\reapply-debloat.ps1
```

EXPECTED RESULT

Lines like `vaulted: ...\ldplayerpartner.exe` and `ad cache sealed (deny Everyone write)`. Then re-run Step 1's verifier and watch every line flip to `[PASS]`.

4 Seal the ad cache

MEDIUM ~3 min risk: low

THE WHY

The cache folder is where the current ad payloads are kept between launches. Emptying alone is not enough — LDPlayer re-downloads them. A deny-write ACE makes the folder permanently useless to it.

WHAT THE SCRIPT DOES

`reapply-debloat.ps1` already performs this — moves contents into the vault, then adds a single Deny ACE: Everyone, Write, inherited by children. Nothing else in the ACL is touched.

> *i* **Verified:** after the seal, a file write to the cache fails with access denied. If it does not, your account is not elevated — re-run as administrator.

5 Re-pin update & telemetry timestamps

MEDIUM ~2 min risk: low

THE WHY

`leidians.config` holds timestamps that tell LDPlayer when to call home and when to look for an update. Its update re-arms these to "now" on every install; the script pins them to 2100 again.

LD · COMMAND

```
powershell -ExecutionPolicy Bypass -File C:\LDPlayer\reapply-debloat.ps1
```

Expected: `leidians.config pins re-applied (present keys only)`. Keys that do not exist in your LDPlayer version are skipped rather than invented.

> *i* **Note:** this step is idempotent and also self-healing — it is re-applied at every run of the reapply script, so an update re-arming the pins is defeated automatically.

6 CPU & RAM tuning — the 3072 rule

HARD | ~10 min + one test boot | risk: medium

THE RULE

Give the guest no more than ~40% of host memory when anything else is running (a browser tab, a game, an IDE) — and no more than ~50% ever. On a memory-tight box that is almost always **2048–4096 MB** depending on hardware. **3072 MB with 3 CPUs is the measured sweet spot for an 8 GB host.**

HOW

The script sets CPU back to 3 with LD's own `ldconsole`:

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\ldconsole.exe" modify --index 0 --cpu 3
```

Memory is set inside `leidian.vbox` (`RAMSize` value) and is written out by LD on launch, so do not edit it while LDPlayer is running.

WHY IT IS THE HARDEST STEP

Machines that are not memory-constrained pass Step 6 painlessly; machines that are will boot-slow or wedge until the memory is lowered. If Boot loops twice identically, reduce by 512 MB and test again.

7 Kill the service

MEDIUM | ~2 min | risk: low

WHY

`LDPlayerENSvr` (name varies by build — LDPlayerENSvc / LDPlayerServiceEN also appear) auto-starts the launcher plumbing. It is not required for the emulator to run; it is the delivery mechanism for the ad channels.

LD · COMMAND

```
Get-Service LDPlayerENSvr -ErrorAction SilentlyContinue | Select-Object Status, StartType
Stop-Service LDPlayerENSvr -Force -ErrorAction SilentlyContinue
Set-Service LDPlayerENSvr -StartupType Disabled
```

The reapply script does the same thing with tolerance for missing services.

8 Network layers: hosts + firewall

MEDIUM | ~4 min | risk: low

LAYER A — marked hosts block

Ten domains map to 0.0.0.0 under a fence. Nothing else in the file changes:

LD · COMMAND

```
$block = @"

# LDPlayer-Debloat-Block-Begin (LDPlayer ad/telemetry endpoints)
0.0.0.0 ldrescdn.com
0.0.0.0 res.ldrescdn.com
0.0.0.0 img.ldrescdn.com
0.0.0.0 tj.ldmnq.com
0.0.0.0 ad.ldmnq.com
0.0.0.0 ads.ldplayer.net
0.0.0.0 report.ldmnq.com
0.0.0.0 alog.ldmnq.com
0.0.0.0 data.ldplayer.net
0.0.0.0 res.ldplayer.net
# LDPlayer-Debloat-Block-End
"@
[IO.File]::AppendAllText("C:\Windows\System32\drivers\etc\hosts", $block, (New-Object
Text.ASCIIEncoding))
```

LAYER B — program-scoped firewall blocks

Five outbound Block rules named `LDPlayerBlock-*`, one per ad/telemetry executable. Because they are scoped by program path, LD's next update cannot rotate the domains past them.

Both layers are applied automatically by `reapply-debloat.ps1`. No extra command is needed.

9 Wave 2 — the deep sweep

MEDIUM ~10 min risk: low

WHY

The "classic" cleanup covers the advertised files; the install root ships a second telemetry stack that classic guides never look at. `reapply-debloat.ps1` removes them too:

- **upload** — a Huawei OBS crash-telemetry SDK (`ldupload.exe` + associated DLLs). Vaulted wholesale; firewall-blocked in Step 8.
- **Manager's bugreport.exe** — a second reporter beside the one in LDPlayer9. Found only by inventoring the whole tree.
- **App-store intercept feed** — `Documents\XuanZhi9\Applications\ld_launcher\` + `cpi.data` + a fetch-stamp (hex-named). Redirects Google Play installs to the LD-store.
- **Recommendation DB** — `vms\recommendConfigs\dir_*.dir` + `package_configs_1005*`. Game suggestions with ad placements.
- **partner_report.data** — the 10-byte file that witnesses the last successful report POST.

Every one of these is re-vaulted automatically at each run of the reapply script.

10 Permanence — the logon enforcement task

HARD ~5 min risk: low

WHY

Updates restore everything you removed — not because they are malicious, because LD ships in one piece and the update is that piece. A logon-triggered hidden task re-runs the reapply script so the debloat survives updates indefinitely.

LD · COMMAND

```
$action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument '-NoProfile -
ExecutionPolicy Bypass -WindowStyle Hidden -File C:\LDPlayer\reapply-debloat.ps1'
$trigger = New-ScheduledTaskTrigger -AtLogOn -User "$env:USERDOMAIN\$env:USERNAME"
$principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME" -RunLevel
Highest -LogonType Interactive
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries -ExecutionTimeLimit (New-TimeSpan -Minutes 5) -Hidden
Register-ScheduledTask -TaskName 'LDPlayerDebloatEnforce' -Action $action -Trigger
$trigger -Principal $principal -Settings $settings -Force
```

PROVE IT

LD · COMMAND

```
Start-ScheduledTask -TaskName 'LDPlayerDebloatEnforce'
(Get-ScheduledTaskInfo -TaskName 'LDPlayerDebloatEnforce').LastTaskResult
```

Expect `0`.

11 Prove it — the verifiers

EASY | ~1 min each | risk: none

WINDOWS SIDE

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\verify-debloat.ps1
```

• • • OUTPUT

```
RESULT: ALL CHECKS PASSED          # exit code 0
```

ANDROID SIDE (run with the emulator up)

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\verify-android.ps1
```

• • • OUTPUT

```
RESULT: ALL ANDROID CHECKS PASSED    # or SKIP if the emulator is not up
```

Both scripts print one line per check and set the exit code to the number of FAILs. A single FAIL names exactly which check failed.

12 Inside Android (ADB)

EXPERT | ~15 min | risk: medium

WHY THIS IS A SEPARATE STEP

Steps 1–11 fix Windows-side plumbing. The LD-store engine lives *inside the guest*, and only answers to ADB. An update can also reset the in-guest changes on re-image, so this step is rerunnable by design.

1 — Enable ADB debugging

LDPlayer → gear icon → Other → **ADB debugging: ON**. If this is off, every command below returns nothing. Every LD update turns it back off.

2 — Install the replacements

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" install -r "C:\ld-apps\Lawnchair.apk"
& "C:\LDPlayer\LDPlayer9\adb.exe" install -r "C:\ld-apps\Aurora.apk"
& "C:\LDPlayer\LDPlayer9\adb.exe" install -r "C:\ld-apps\Magisk.apk"
```

Pickers for the APKs: Lawnchair 1.2.0 → `github.com/LawnchairLauncher/lawnchair/releases` · Aurora Store → `f-droid.org/packages/com.aurora.store` · Magisk → `github.com/topjohnwu/Magisk/releases`. Verify package names with `aapt dump badging` before installing anything.

3 — Disable LD's own apps (matched by name pattern)

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell cmd package disable-user --user 0
com.ldmnq.launcher3
& "C:\LDPlayer\LDPlayer9\adb.exe" shell cmd package disable-user --user 0
com.android.ld.appstore
```

4 — Home screen

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell cmd package set-home-activity
ch.deletescape.lawnchair/ch.deletescape.lawnchair.Launcher
```

5 — Ad-blocking DNS

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell settings put global private_dns_mode hostname
& "C:\LDPlayer\LDPlayer9\adb.exe" shell settings put global private_dns_specifier
dns.adguard.com
```

6 — The one step that is yours alone

Magisk → Install → Direct install → Reboot. Until this is done, `su -c id` correctly reports root not live.

13 Live clean forever

EASY | ~2 min per update | risk: none

THE ROUTINE

1. Open LDPlayer exclusively via the **Clean Launch** shortcut, never the stock icon.
2. Close what you are not actively using — LD will take whatever memory it is left.
3. After any LDPlayer update: log off and back on, then run both verifiers. Any FAIL is fixed by re-running `reapply-debloat.ps1` and `android-refresh.ps1`.
4. Thirty quiet days in a row: delete `C:\LDPlayer\vault` to reclaim disk.

THE CLEAN LAUNCH SHORTCUT

`clean-launch.ps1` debloats, launches LDPlayer, then re-sweeps 75 seconds later — after the launch-time re-download lands. One double-click, nothing else.

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\clean-launch.ps1
```

Make it a shortcut on the desktop (right-click → New → Shortcut → paste `powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File C:\LDPlayer\clean-launch.ps1`) and bind the stock LDPlayer icon to it.

WHEN THINGS BREAK

Errors & fixes 06

The two classes (learn to tell them apart)

! **CLASS A — Deterministic:** same spot, same signature, two times in a row (an admin refusal; a boot wedge at the same second). It never fixes itself. **Change the variable, not your mood.**

i **CLASS B — Load-dependent:** only happens when the machine is saturated (a game, a browser, an IDE and LD competing for the same memory). **It is not a bug; it is arithmetic.** Step 6 exists for exactly this class.

The field list

#	WHAT YOU SEE	CAUSE	FIX
1	`run player with no Admin error` in logs / LDPlayer silently refuses to start	The engine requires elevation	Launch LDPlayer **as administrator**
2	Boot freezes at roughly the same second, twice	Guest RAM too small for first boot	Raise to 3072 MB; re-test; boot×2 identical → reduce and re-test

#	WHAT YOU SEE	CAUSE	FIX
3	"Not Responding", fan noise, stutter	Host starved: the guest reservation exceeds the host's surplus	Cap the guest at 3072 MB · close what you do not need · re-run Step 6
4	`adb devices` empty with Android visibly running	ADB debugging flag is off	LDPlayer → gear → Other → **ADB debugging: ON**
5	"Access is denied" writing into `%APPDATA%\XuanZhi9\cache`	ACL deny ACE got removed or corrupt	Re-run `reapply-debloat.ps1` as administrator; step 2 re-seals
6	"This app can't run on your PC" / blank window on launch	Instance's CPU count became 4 on a starved host	`ldconsole modify --index 0 --cpu 3`
7	Verifier prints garbage for HOME resolution	Command-line flag confusion on Android 9	`cmd package resolve-activity --brief … HOME` directly; trust `isDefault=true`
8	`VERR_*` errors or VM "not responding"	Hand-edited `leidian.vbox` with the emulator running	Never edit it live; quit LDPlayer, edit, relaunch; check `vms\leidian0\Logs\VBox.log` tail
9	Everything reverted after an update	Updates re-ship their own bloat	Sign out and back in — the logon task re-fixes; verify with `verify-debloat.ps1`
10	Task Scheduler reports the task ran but FAIL persists	Path changed or task lacked highest privileges	Re-create the task with `-RunLevel Highest`
11	Ads still visible in the guest	In-guest LD store installed / LD launcher enabled	Re-run Step 12 (or `android-refresh.ps1`)
12	Update check keeps offering builds	`nextCheckupdateTime` re-armed to now	Re-run `reapply-debloat.ps1`; it pins to 2100
13	LDPlayer refuses to `repair` / "installer not found"	`dnrepairer.exe` missing (this guide removes it)	There is no repair. The install is either stable or reinstalled from the original installer
14	`adb kill-server` refused; port 5037 in use	Another ADB server on this machine	`& "C:\LDPlayer\LDPlayer9\adb.exe" kill-server` then relaunch LDPlayer
15	Verifier exits non-zero	One or more FAIL lines — that is the exit code's meaning	Read each FAIL; run `reapply-debloat.ps1`; re-check
16	Ads/recommendations reappear between reboots	LD re-downloads its ad feed ~2 minutes after every launch	The **Clean Launch** shortcut watches and re-vaults it; or run `reapply-debloat.ps1` after every session
17	`su: not found` despite Magisk installed	Boot image is unpatched	Magisk → Install → Direct install → Reboot
18	ADB debugging was off again after an update	Updates turn it off	Gear → Other → ADB debugging: ON (re-check after every update)
19	Home screen shows LD launcher after an update	The update re-imaged the guest	Re-run Step 12 (or `android-refresh.ps1`)

THE TOOLKIT

Your new toolkit 07

All five files ship in this guide's repository under `scripts/`. Copy them to `C:\LDPlayer\`. They find your install automatically.

FILE	DOES WHAT	RUN IT WHEN
`C:\LDPlayer\reapply-debloat.ps1`	Master fixer: vaults restored bloat, re-seals the cache, re-pins 2100, disables the service, re-ensures hosts + firewall, and (when the emulator is attached) re-applies the Android side. Self-elevates. Idempotent.	After every update, after every error, or whenever the verifier reports FAIL. Safe to run any time.
`C:\LDPlayer\clean-launch.ps1`	Debloat → launch LDPlayer (index 0) → detached watcher re-sweeps ~75 s later	**Every launch**, instead of the stock icon
`C:\LDPlayer\android-refresh.ps1`	Inside-Android re-apply: LD packages disabled (pattern-matched), Private DNS=adguard, Lawnchair re-set as HOME	After any update that re-imaged the guest, or if the guest looks wrong
`C:\LDPlayer\verify-debloat.ps1`	Read-only audit (Windows side). One line per check, exit code = number of FAILs.	After any update
`C:\LDPlayer\verify-android.ps1`	Read-only audit (Android side). One line per check, exit code = number of FAILs.	After any update, with the emulator running
Scheduled task `LDPlayerDebloatEnforce`	Runs the reapply script at every logon, hidden, highest privileges	Never run it by hand — that is the point

What each command prints

LD · COMMAND

```
RESULT: ALL CHECKS PASSED                    # verify-debloat.ps1 — exit code 0
RESULT: 0 problem(s) found                 # or, failing that, the named checks
RESULT: ALL ANDROID CHECKS PASSED          # verify-android.ps1 — or SKIP if emulator is
down
```

The folder map

LD · COMMAND

```
C:\LDPlayer\
  ├─ LDPlayer9\            ← your install (or LDPlayer12 / LDPlayer5 / ...)
  ├─ ldmutiplayer\         ← multi-instance manager
  ├─ vault\                ← everything that was moved here, never deleted
  ├─ reapply-log.txt       ← every reapply run, timestamped
  ├─ android-refresh-log.txt ← every Android-side run, timestamped
  └─ reapply-debloat.ps1 / clean-launch.ps1 / android-refresh.ps1 / verify-debloat.ps1 /
verify-android.ps1

%APPDATA%\XuanZhi9\cache  ← sealed; holds nothing
Documents\XuanZhi9\Applications ← LD-store staging; removed
```

SCOPE SUMMARY

Difficulty × time × risk

08

STEP	DO WHAT	DIFFICULTY	TIME	RISK
0	Symptom check	EASY	1 min	none
1	Install toolkit	EASY	2 min	none
2	Photo evidence	EASY	5 min	none
3	Vault classic bloat	EASY	3 min	none
4	Seal the ad cache	MEDIUM	3 min	low
5	Re-pin timestamps	MEDIUM	2 min	low
6	Size guest RAM/CPU	HARD	10 min	medium
7	Disable the service	MEDIUM	2 min	low
8	Hosts + firewall layers	MEDIUM	4 min	low
9	Deep sweep (upload SDK + hijack feed)	HARD	10 min	low
10	Logon enforcement task	HARD	5 min	low
11	Verify (both verifiers)	EASY	2 min	none
12	Inside Android (ADB)	EXPERT	15 min	medium
13	Living clean	EASY	2 min per update	none

Total elbow-grease: about 60–90 minutes once, then ~2 minutes per LDPlayer update, forever.

SPEAK THE LANGUAGE

Glossary

09

A–C

ACE — a single permission rule inside a file/folder's ACL. We use one **deny-write** ACE.
ACL — the rule list that says who may read, write or run an object.
ADB — Android Debug Bridge: the wire that lets your PC talk into the emulator.
Aurora Store — clean front-end for Play.
App-store intercept — LD-store redirect rules for Google Play installs (Class 2).

D–H

dnplayer — LDPlayer's engine (VirtualBox-based).
Disable-user — Android's drop-in replacement for "uninstall a system app for this user".
Direct install — Magisk's button that patches the boot image; without it root does not exist.
Emulator — the running Android inside LDPlayer.
exit code — a number a program returns; 0 = fine.

I–L

Idempotent — running it again changes nothing.
interceptTiming — inside LD's JSON: *when* to hijack.
LD-store — LD's own app store; where intercepts are sent.
Launcher — the Android home-screen app (Lawnchair is the replacement).

M–Z

Magisk — root manager.
Private DNS — forces DNS through your chosen (ad-blocking) resolver.
Private DNS specifier — the hostname of that resolver (`dns.adguard.com`).
Recommendation DB — `dir_*.dir`: "great games for you" with ads.
The pin — the far-future timestamps in `leidians.config`.
ldmarket:// — the LD-store URI scheme.
VBox.log — the VM's flight recorder; read it when boots fail.

YOUR QUESTIONS, ANSWERED

FAQ 10

Will this make my games faster?

Indirectly. The elimination of ad processes, telemetry and re-downloads removes background drains; the 3072 MB/3-CPU rule stops swap when other apps are running; removing the intercept feed means your apps come from Play, not from LD's clone. None of those are frame-rate tools, and none promise you will suddenly get 240 FPS.

Is this legal? Did we crack anything?

No patching, repacking or DRM circumvention happens anywhere. We move extra files (reversible), flip LDPlayer's own settings, and use standard Windows firewall/hosts rules. It runs LDPlayer with fewer passengers, not a different LDPlayer.

Will LDPlayer still update?

LD always ships as one piece; any update reintroduces the bloat. The Clean Launch watcher and the logon task re-apply the debloat automatically. Disabling the in-guest feed means the update itself installs faster and draws less.

Why the one firewall rule per executable instead of blocking domains?

Domains rotate; executable paths do not, until the next LD build reinstalls them — at which point the old rules silently re-apply (path unchanged) and you win again without any new work.

My friend says "everything came back after the update". Why?

Because their cleanup ran once, as a single pass. This is why Step 10 and Step 13 exist — the fix is a loop, not an event.

I'm not technical. Which steps can I skip?

Steps 0–11 are mechanical. Never skip Step 12 if you care about the in-guest ads, and Step 10 if you care about updates keeping it that way.

Magisk asked to Direct install; do I have to?

Yes — Magisk's binary lives in the boot image; the manager alone does not patch it. Without Direct install, `su` exists but root never fires.

Can LDPlayer detect that I debloated it?

Yes, in the same sense it can detect you removed an uninstaller. It does not phone home about it, and it does not lock features. The bloat is gone, LD still runs, support still works.

Where is LD's official FAQ for all this?

There is none. What you just read was produced by doing it carefully, watching what came back, and keeping what proved necessary.

THE HOST OS

Windows 10 Lite — what is added, what is not 11

Some Windows builds are thinned by third parties ("Lite", "Tiny10", handcrafted ISOs). This guide never touches those' Windows: no system files, no shell, no services outside LDPlayer's, no removal of anything host-side except LDPlayer's own binaries.

Exactly two additions (both reversible, both documented on screen)

ADDITION	WHERE	WHY IT EXISTS
Markers: one fenced hosts block (10 addresses)	`C:\Windows\System32\drivers\etc\hosts`	Blocks the ad/telemetry domains LD answers on launch
Firewall outbound blocks named `LDPlayerBlock-*`	Windows Firewall	Blocks the exact executables, whatever domain they rotate to

What is intentionally NOT done

- Defender / Exclusions: not touched.
- Lite-specific components (WiFi, Edge, nowhere-near-essentials): not touched.
- Power plan, services, scheduled tasks outside LDPlayer's own: not touched.
- Your user folders, OneDrive, game saves: not touched.

> *i* **The rule of thumb:** if the change could be explained on a post-it in one sentence about LDPlayer, it is fair game. Anything else belongs to a different project.

THE MODEL

Why it survives updates 12

LDPlayer is one piece of software that re-ships its full runtime on every update. When the update runs, it copies in whatever it ships — bloat and all — and re-enables what you disabled. That is not a hostile fork; it is simply how LDPlayer is built. So the fix is not an event; it is a **standing invariant**:

★ **The three-move loop:** (1) at logon, a hidden task re-vaults anything restored and re-pins; (2) at launch, the Clean Launch shortcut re-vaults whatever re-downloaded during startup; (3) both verifiers tell you whether the invariant held. That is the entire permanence model.

What survives, what does not

SURVIVES	DOES NOT SURVIVE
Windows-internal edits: hosts block, firewall rules, service start type	Guest-internal edits (packages disabled, HOME setting) — they live in `data.vmdk` and survive updates unless the update re-images it
The pinned update timestamps	The launch-time feed download — it re-arrives on every launch until the watcher re-vaults it
The seal on the cache folder	The installer stub — it always re-appears; the reapply script always removes it

How to keep the invariant

1. Never launch LDPlayer via the stock icon. Launch via the Clean Launch shortcut.
2. After every update, log off and back on. The logon task fires.
3. Run both verifiers. They tell you whether the loop held.

PRINT THIS PAGE

Final checklist 13

Part A — Windows side

- ☐ LDPlayer launched only via **Clean Launch**
- ☐ `verify-debloat.ps1` prints **ALL CHECKS PASSED**
- ☐ Ad cache exists, empty, deny-write sealed
- ☐ `leidians.config`: telemetry pinned to 2100
- ☐ `leidian0.config`: `advancedSettings.cpuCount = 3`
- ☐ LDPlayer's service: **Stopped + Disabled**
- ☐ hosts: one marked block, nothing else
- ☐ Firewall: `LDPlayerBlock-*` all present
- ☐ In `Documents\XuanZhi9\Applications`: no `ld_launcher`, no `cpi.data`, nothing hex-named
- ☐ In `vms\recommendConfigs`: only `dynamic-key.xml` remains
- ☐ Brave/other heavy apps closed while playing

Part B — Inside Android

- ☐ `verify-android.ps1` prints **ALL ANDROID CHECKS PASSED**
- ☐ Home screen: Lawnchair
- ☐ `pm list packages -d` includes LD launcher / LD app-store
- ☐ Private DNS: `dns.adguard.com`

- [ ] Magisk Direct install done; `su -c id` prints uid=0
- [ ] No LD-store redirect when installing from Play

Part C — Your job after this

- [ ] Every launch of LDPlayer: via the Clean Launch shortcut
- [ ] After every update: log off/on, then verify
- [ ] Thirty days later: delete `C:\LDPlayer\vault` and reclaim the space

✓ **When every box is ticked,** you are running a stock LDPlayer with its advertising removed — nothing else changed, nothing hidden, nothing broken.

THE FILES, VERBATIM

Appendix: the scripts 14

Every file this guide creates is reproduced here in full, exactly as it ships. Copy & paste into a new file. One-time, about ten minutes, and you will never need to write any of it by hand.

reapply-debloat.ps1

Master re-fixer. Idempotent. Self-elevating.

LD · COMMAND

```
# reapply-debloat.ps1 - Re-applies the LDPlayer debloat. Safe to run any number of times.
# Works for any LDPlayer version installed at the usual locations. Self-elevates to admin.
# Everything it moves goes into C:\LDPlayer\vault. Nothing is deleted.
# If auto-detection fails, set $LD_ROOT by hand (the folder that contains ldconsole.exe).
$LD_ROOT = $null
foreach ($candidate in
@('C:\LDPlayer\LDPlayer9','C:\LDPlayer\LDPlayer12','C:\LDPlayer\LDPlayer5','C:\LDPlayer\LDPlay
er4','C:\LDPlayer\LDPlayer9x')) {
  if (Test-Path "$candidate\ldconsole.exe") { $LD_ROOT = $candidate; break }
}
if (-not $LD_ROOT) {
  $hit = Get-ChildItem 'C:\LDPlayer' -Directory -Force -ErrorAction SilentlyContinue | Where-
Object { Test-Path (Join-Path $_.FullName 'ldconsole.exe') } | Select-Object -First 1
  if ($hit) { $LD_ROOT = $hit.FullName }
}
if (-not $LD_ROOT) { Write-Host 'ERROR: no LDPlayer install with ldconsole.exe found under
C:\LDPlayer. Set $LD_ROOT manually.'; exit 2 }

$Log = 'C:\LDPlayer\reapply-log.txt'
function Log($m){
  $line = "$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')  $m"
  Add-Content -Path $Log -Value $line -Encoding ASCII
  Write-Host $line
}
Log "using install root: $LD_ROOT"
$LD_PACKAGE = "$LD_ROOT\..\ldmutiplayer"
if (-not (Test-Path $LD_PACKAGE)) { $LD_PACKAGE = $null }

# --- self-elevate if needed ---
$id = [Security.Principal.WindowsIdentity]::GetCurrent()
$p  = New-Object Security.Principal.WindowsPrincipal($id)
if (-not $p.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
  Log 'not elevated; relaunching as administrator...'
  Start-Process powershell.exe -Verb RunAs -ArgumentList "-NoProfile -ExecutionPolicy Bypass -
File `"$PSCommandPath`""
  exit
}

Log '=== reapply-debloat start ==='

# --- 1) vault restored bloat ---
$vault = Join-Path 'C:\LDPlayer\vault' ("after-" + (Get-Date -Format 'yyyyMMdd-HHmmss'))
$candidates = @(
  "$LD_ROOT\ldplayerpartner.exe",
  "$LD_ROOT\ldplayerpartner2.exe",
  "$LD_ROOT\bugreport.exe",
  "$LD_ROOT\dnrepairer.exe",
  "$LD_ROOT\LDPlayer_*.exe",
  "$LD_ROOT\log\ldpartner.log"
)
if ($LD_PACKAGE) { $candidates += "$LD_PACKAGE\bugreport.exe" }
foreach ($item in $candidates) {
  $parent = Split-Path $item -Parent
  $leaf   = Split-Path $item -Leaf
  if (Test-Path $parent) {
    Get-ChildItem $parent -Filter $leaf -Force -ErrorAction SilentlyContinue | ForEach-Object
{
      $dest = Join-Path $vault (Split-Path $_.DirectoryName -Leaf)
      New-Item -ItemType Directory -Force -Path $dest | Out-Null
      Move-Item $_.FullName (Join-Path $dest $_.Name) -Force
      Log "vaulted: $($_.FullName)"
    }
```

```
  }
}

# --- 1b) crash-telemetry SDK re-vaulted ---
$upload = Join-Path $LD_ROOT 'upload'
if (Test-Path $upload) { Move-Item $upload (Join-Path $vault 'upload') -Force; Log 'vaulted:
telemetry SDK (upload\)' }

# --- 1c) app-store intercept/ad staging feed re-vaulted ---
$appStore = $null
foreach ($cand in
@("$env:USERPROFILE\Documents\XuanZhi9\Applications","$env:USERPROFILE\Documents\XuanZhi\Appli
cations","$env:USERPROFILE\Documents\Letu\Applications")) {
  if ($cand -and (Test-Path $cand)) { $appStore = $cand; break }
}
if ($appStore) {
  foreach ($item in @('ld_launcher','cpi.data','appInfo.txt')) {
    $src = Join-Path $appStore $item
    if (Test-Path $src) {
      $dest = Join-Path $vault 'intercept'; New-Item -ItemType Directory -Force -Path $dest |
Out-Null
      Move-Item $src (Join-Path $dest (Split-Path $src -Leaf)) -Force
      Log "vaulted: $src"
    }
  }
  Get-ChildItem $appStore -Force -ErrorAction SilentlyContinue | Where-Object { $_.Name -match
'^[0-9a-f]{32}$' } | Move-Item -Destination (Join-Path $vault 'intercept') -Force
}

# --- 1d) recommendation/ad DB re-vaulted ---
$rec = "$LD_ROOT\vms\recommendConfigs"
if (Test-Path $rec) {
  foreach ($item in
@('dir_kmps.dir','dir_jmps.dir','dir_noices.dir','dir_commons.dir','package_configs_1005','pac
kage_configs_1005.rcc')) {
    $src = Join-Path $rec $item
    if (Test-Path $src) {
      $dest = Join-Path $vault 'recommendConfigs'; New-Item -ItemType Directory -Force -Path
$dest | Out-Null
      Move-Item $src (Join-Path $dest $item) -Force
      Log "vaulted: $src"
    }
  }
}

# --- 2) purge + re-seal ad cache ---
$cache = $null
foreach ($cand in
@("$env:APPDATA\XuanZhi9\cache","$env:APPDATA\XuanZhi\cache","$env:APPDATA\Letu\cache")) {
  if (Test-Path $cand) { $cache = $cand; break }
}
if ($cache) {
  $pending = Get-ChildItem $cache -Force -ErrorAction SilentlyContinue
  if ($pending) {
    $cd = Join-Path $vault 'adcache'; New-Item -ItemType Directory -Force -Path $cd | Out-Null
    $pending | Move-Item -Destination $cd -Force
    Log 'ad cache contents vaulted'
  }
  icacls $cache /remove:d Everyone 2>&1 | Out-Null
  icacls $cache /deny 'Everyone:(OI)(CI)W' 2>&1 | Out-Null
  Log 'ad cache sealed'
} else { Log 'no ad cache folder found - skipping seal' }

# --- 3) re-pin update/telemetry pins (handles missing keys gracefully) ---
```

```
$cfgPath = "$LD_ROOT\vms\config\leidians.config"
if (Test-Path $cfgPath) {
  try {
    $cfg = Get-Content $cfgPath -Raw | ConvertFrom-Json
    foreach ($k in @('reportPCInfoTime','adLoadtime','nextCheckupdateTime')) {
      if ($cfg.PSObject.Properties.Name -contains $k) {
        if ($k -eq 'reportPCInfoTime') { $cfg.$k = '20990101' } else { $cfg.$k = 4102444800 }
      }
    }
    if ($cfg.PSObject.Properties.Name -contains 'noiceUserRed') { $cfg.noiceUserRed = $false }
    [System.IO.File]::WriteAllText($cfgPath, ($cfg | ConvertTo-Json -Depth 5), (New-Object
System.Text.ASCIIEncoding))
    Log 'leidians.config pins re-applied (present keys only)'
  } catch { Log "leidians.config patch failed: $($_.Exception.Message)" }
} else { Log 'leidians.config not found - skipping config pins' }

# --- 4) CPU back to 3 via official CLI ---
$ldconsole = "$LD_ROOT\ldconsole.exe"
if (Test-Path $ldconsole) {
  & $ldconsole modify --index 0 --cpu 3 2>&1 | Out-String | ForEach-Object { Log "ldconsole:
$_" }
}

# --- 5) telemetry/ad service disabled ---
foreach ($svcName in @('LDPlayerENSvr','LDPlayerENSvc','LDPlayerServiceEN')) {
  try {
    $svc = Get-Service $svcName -ErrorAction Stop
    if ($svc.Status -ne 'Stopped') { Stop-Service $svcName -Force }
    Set-Service $svcName -StartupType Disabled
    Log "$svcName stopped + disabled"
  } catch { Log "$svcName not found (ok)" }
}

# --- 6) hosts block: ensure present ---
$hostsPath = 'C:\Windows\System32\drivers\etc\hosts'
$hostsRaw  = Get-Content $hostsPath -Raw
if ($hostsRaw -notmatch 'LDPlayer-Debloat-Block-Begin') {
  $block = @'

# LDPlayer-Debloat-Block-Begin (LDPlayer ad/telemetry endpoints)
0.0.0.0 ldrescdn.com
0.0.0.0 res.ldrescdn.com
0.0.0.0 img.ldrescdn.com
0.0.0.0 tj.ldmnq.com
0.0.0.0 ad.ldmnq.com
0.0.0.0 ads.ldplayer.net
0.0.0.0 report.ldmnq.com
0.0.0.0 alog.ldmnq.com
0.0.0.0 data.ldplayer.net
0.0.0.0 res.ldplayer.net
# LDPlayer-Debloat-Block-End
'@
  [System.IO.File]::AppendAllText($hostsPath, $block, (New-Object System.Text.ASCIIEncoding))
  Log 'hosts block applied'
} else { Log 'hosts block already present' }

# --- 7) firewall rules (only for files that actually exist) ---
$rules = @{
  'LDPlayerBlock-ldplayerpartner'   = "$LD_ROOT\ldplayerpartner.exe"
  'LDPlayerBlock-ldplayerpartner2'  = "$LD_ROOT\ldplayerpartner2.exe"
  'LDPlayerBlock-dnrepairer'        = "$LD_ROOT\dnrepairer.exe"
  'LDPlayerBlock-bugreport'         = "$LD_ROOT\bugreport.exe"
  'LDPlayerBlock-ldplayerserviceEN' = "$LD_ROOT\ldplayerserviceEN.exe"
  'LDPlayerBlock-ldupload'          = "$LD_ROOT\upload\ldupload.exe"
```

```
}
if ($LD_PACKAGE) { $rules['LDPlayerBlock-bugreport-manager'] = "$LD_PACKAGE\bugreport.exe" }
foreach ($name in $rules.Keys) {
  $prog = $rules[$name]
  if (Test-Path $prog) {
    if (-not (Get-NetFirewallRule -DisplayName $name -ErrorAction SilentlyContinue)) {
      New-NetFirewallRule -DisplayName $name -Direction Outbound -Action Block -Program $prog
-Profile Any -Enabled True | Out-Null
      Log "firewall rule created: $name"
    }
  }
}

# --- 8) in-Android state (only if the emulator is running right now) ---
$adb = "$LD_ROOT\adb.exe"
if (Test-Path $adb) {
  $out = (& $adb devices | Out-String)
  if ($out -match 'emulator-\d+\s+device') {
    Log 'adb: device attached - re-applying in-Android debloat'
    & 'C:\LDPlayer\android-refresh.ps1' | ForEach-Object { Log "  android: $_" }
  } else { Log 'adb: emulator not attached (Android state persists in its data image)' }
}

Log '=== reapply-debloat done ==='
```

clean-launch.ps1

Launch LDPlayer and re-sweep 75 seconds later (catches the launch-time re-download).

LD · COMMAND

```
# clean-launch.ps1 - LD purge, launch, then watcher sweep ~75s later. Run it instead of the
stock shortcut.
# Self-elevates. Finds your LDPlayer folder automatically. -WatchOnly is internal; do not run
it directly.
param([switch]$WatchOnly)
$LD_ROOT = $null
foreach ($candidate in
@('C:\LDPlayer\LDPlayer9','C:\LDPlayer\LDPlayer12','C:\LDPlayer\LDPlayer5','C:\LDPlayer\LDPlay
er4','C:\LDPlayer\LDPlayer9x')) {
  if (Test-Path "$candidate\ldconsole.exe") { $LD_ROOT = $candidate; break }
}
if (-not $LD_ROOT) {
  $hit = Get-ChildItem 'C:\LDPlayer' -Directory -Force -ErrorAction SilentlyContinue | Where-
Object { Test-Path (Join-Path $_.FullName 'ldconsole.exe') } | Select-Object -First 1
  if ($hit) { $LD_ROOT = $hit.FullName }
}
$reapply = 'C:\LDPlayer\reapply-debloat.ps1'
$log = 'C:\LDPlayer\reapply-log.txt'
function Log($m){ Add-Content $log "$(Get-Date -f 'yyyy-MM-dd HH:mm:ss')  $m"; Write-Host $m }

if (-not $LD_ROOT) { Write-Host 'ERROR: LDPlayer install not found. Set LD_ROOT in clean-
launch.ps1.'; exit 2 }

# --- self-elevate ---
$id = [Security.Principal.WindowsIdentity]::GetCurrent()
$p  = New-Object Security.Principal.WindowsPrincipal($id)
if (-not $p.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
  $a = if ($WatchOnly) { '-WatchOnly' } else { '' }
  Start-Process powershell.exe -Verb RunAs -ArgumentList "-NoProfile -ExecutionPolicy Bypass -
WindowStyle Hidden -File `"$PSCommandPath`" $a"
  exit
}

if ($WatchOnly) {
  Log 'post-launch watcher: waiting 75s...'
  Start-Sleep -Seconds 75
  & $reapply | ForEach-Object { }
  Log 'post-launch sweep done'
  exit 0
}

Log "=== clean-launch via $LD_ROOT ==="
& $reapply | Out-Null
Log "launching LDPlayer (index 0)..."
& "$LD_ROOT\ldconsole.exe" launch --index 0
Log 'spawning detached watcher'
Start-Process powershell.exe -ArgumentList "-NoProfile -ExecutionPolicy Bypass -WindowStyle
Hidden -File `"$PSCommandPath`" -WatchOnly" -WindowStyle Hidden
Log '=== clean-launch done ==='
```

android-refresh.ps1

Inside-Android re-apply (LD packages disabled, Private DNS, Lawnchair HOME).

LD · COMMAND

```
# android-refresh.ps1 - Re-applies the INSIDE-ANDROID debloat over ADB. Idempotent.
# Requires: the emulator running, and its "ADB debugging" switch ON (Other tab).
$LD_ROOT = $null
foreach ($candidate in
@('C:\LDPlayer\LDPlayer9','C:\LDPlayer\LDPlayer12','C:\LDPlayer\LDPlayer5','C:\LDPlayer\LDPlay
er4','C:\LDPlayer\LDPlayer9x')) {
  if (Test-Path "$candidate\adb.exe") { $LD_ROOT = $candidate; break }
}
if (-not $LD_ROOT) {
  $hit = Get-ChildItem 'C:\LDPlayer' -Directory -Force -ErrorAction SilentlyContinue | Where-
Object { Test-Path (Join-Path $_.FullName 'adb.exe') } | Select-Object -First 1
  if ($hit) { $LD_ROOT = $hit.FullName }
}
$adb = "$LD_ROOT\adb.exe"
$log = 'C:\LDPlayer\android-refresh-log.txt'
function Log($m){ $l = "$(Get-Date -f 'yyyy-MM-dd HH:mm:ss')  $m"; Add-Content $log $l; Write-
Host $l }

$out = (& $adb devices | Out-String)
if ($out -notmatch 'emulator-\d+\s+device') { Log 'FAIL: no emulator visible via adb (is ADB
debugging ON?)'; exit 3 }
Log "adb: device attached ($LD_ROOT)"

# 1) Private DNS (ad-blocking resolver)
& $adb shell settings put global private_dns_mode hostname  | Out-Null
& $adb shell settings put global private_dns_specifier dns.adguard.com | Out-Null
$dns = (& $adb shell settings get global private_dns_specifier | Out-String).Trim()
Log "private DNS: $dns"

# 2) disable LD's Android packages (pattern-matched, not hardcoded)
$patterns =
@('launcher3','ld.appstore','printspooler','dreams.phototable','ld.market','ldmnq')
$all = (& $adb shell pm list packages | Out-String) -replace 'package:','' -split "`r?`n" |
ForEach-Object { $_.Trim() } | Where-Object { $_ }
foreach ($p in $all) {
  foreach ($pat in $patterns) {
    if ($p -match [regex]::Escape($pat)) {
      $res = (& $adb shell pm disable-user --user 0 $p 2>&1 | Out-String).Trim()
      Log "disable-user $p -> $res"
    }
  }
}

# 3) Lawnchair back as HOME if installed
$home = (& $adb shell cmd package query-activities -a android.intent.action.MAIN -c
android.intent.category.HOME | Out-String)
if ($home -match 'ch\.deletescape\.lawnchair') {
  & $adb shell cmd package set-home-activity
ch.deletescape.lawnchair/ch.deletescape.lawnchair.Launcher | Out-Null
  Log 'HOME set: Lawnchair'
} else {
  Log 'Lawnchair not installed - install it first (see guide Step 12)'
}

# 4) report
$disabled = (& $adb shell pm list packages -d | Out-String) -replace "`r",''
Log 'disabled packages:'; $disabled -split "`n" | ForEach-Object { Log "  $_" }
Log '=== android-refresh done ==='
```

verify-debloat.ps1

Read-only audit. Exit code = number of FAILs.

LD · COMMAND

```
# verify-debloat.ps1 - READ-ONLY audit of the LDPlayer install. Prints PASS/FAIL per item.
# Exit code = number of FAILs. Works for any LDPlayer version in the usual locations.
$LD_ROOT = $null
foreach ($candidate in
@('C:\LDPlayer\LDPlayer9','C:\LDPlayer\LDPlayer12','C:\LDPlayer\LDPlayer5','C:\LDPlayer\LDPlay
er4','C:\LDPlayer\LDPlayer9x')) {
  if (Test-Path "$candidate\ldconsole.exe") { $LD_ROOT = $candidate; break }
}
if (-not $LD_ROOT) {
  $hit = Get-ChildItem 'C:\LDPlayer' -Directory -Force -ErrorAction SilentlyContinue | Where-
Object { Test-Path (Join-Path $_.FullName 'ldconsole.exe') } | Select-Object -First 1
  if ($hit) { $LD_ROOT = $hit.FullName }
}
$fails = 0
function Report($l){ Write-Host $l }
function Verify($name, [scriptblock]$test){
  $ok = $false
  try { $ok = [bool](& $test) } catch { $ok = $false }
  if ($ok) { Report "[PASS] $name" } else { Report "[FAIL] $name"; $script:fails++ }
}

if (-not $LD_ROOT) { Report 'FAIL: LDPlayer install not found'; exit 2 }
Report "LDPlayer: $LD_ROOT"

$ver = (Get-Item "$LD_ROOT\dnplayer.exe" -ErrorAction
SilentlyContinue).VersionInfo.ProductVersion
Report "version: $ver"

foreach ($bloat in
@('ldplayerpartner.exe','ldplayerpartner2.exe','bugreport.exe','dnrepairer.exe')) {
  Verify "$bloat absent" { -not (Test-Path (Join-Path $LD_ROOT $bloat)) }
}
Get-ChildItem $LD_ROOT -Filter 'LDPlayer_*.exe' -ErrorAction SilentlyContinue | ForEach-Object
{ Verify "stub absent: $($_.Name)" { $false } }
Verify 'no Huawei OBS upload SDK (upload\)'      { -not (Test-Path "$LD_ROOT\upload") }
Verify 'partner log absent'                      { -not (Test-Path
"$LD_ROOT\log\ldpartner.log") }

$appStore = $null
foreach ($cand in
@("$env:USERPROFILE\Documents\XuanZhi9\Applications","$env:USERPROFILE\Documents\XuanZhi\Appli
cations","$env:USERPROFILE\Documents\Letu\Applications")) {
  if ($cand -and (Test-Path $cand)) { $appStore = $cand; break }
}
Verify 'app-store ad feed absent (cpi.data)'    { $appStore -eq $null -or -not (Test-Path
(Join-Path $appStore 'cpi.data')) }
Verify 'app-store intercept feed absent'        { $appStore -eq $null -or -not (Test-Path
(Join-Path $appStore 'ld_launcher')) }

$rec = "$LD_ROOT\vms\recommendConfigs"
$recLeft = if (Test-Path $rec) { Get-ChildItem $rec -Force -ErrorAction SilentlyContinue |
Where-Object { $_.Name -ne 'dynamic-key.xml' } } else { $null }
Verify 'recommendation/ad DB vaulted'           { -not $recLeft }

$cache = $null
foreach ($cand in
@("$env:APPDATA\XuanZhi9\cache","$env:APPDATA\XuanZhi\cache","$env:APPDATA\Letu\cache")) {
  if (Test-Path $cand) { $cache = $cand; break }
}
Verify 'ad cache exists and is empty'           { $cache -and -not (Get-ChildItem $cache -
Force -ErrorAction SilentlyContinue) }
Verify 'ad cache sealed (write denied)'         { $cache -and [bool]((Get-Acl $cache).Access |
```

```
Where-Object { $_.AccessControlType -eq 'Deny' -and $_.IdentityReference -like '*Everyone*' -
and $_.FileSystemRights.ToString() -like '*Write*' }) }

$cfgPath = "$LD_ROOT\vms\config\leidians.config"
if (Test-Path $cfgPath) {
  $cfg = Get-Content $cfgPath -Raw | ConvertFrom-Json
  Verify 'telemetry report pinned'                { ($cfg.reportPCInfoTime -eq '20990101') -or
(-not $cfg.reportPCInfoTime) }
  Verify 'adLoadtime pinned to 2100'              { ($null -eq $cfg.adLoadtime) -or
($cfg.adLoadtime -ge 4102444800) }
  Verify 'update check pinned to 2100'            { ($null -eq $cfg.nextCheckupdateTime) -or
($cfg.nextCheckupdateTime -ge 4102444800) }
}
$vbox = Join-Path $LD_ROOT 'vms\leidian0\leidian.vbox'
if (Test-Path $vbox) {
  $v = Get-Content $vbox -Raw
  Verify 'RAM cap = 3072'                         { $v -match 'RAMSize="3072"' }
}

foreach ($svcName in @('LDPlayerENSvr','LDPlayerENSvc','LDPlayerServiceEN')) {
  $svc = Get-Service $svcName -ErrorAction SilentlyContinue
  if ($svc) { Verify "$svcName disabled" { $svc.StartType -eq 'Disabled' } }
}

Verify 'hosts block present'                      { (Get-Content
'C:\Windows\System32\drivers\etc\hosts' -Raw) -match 'LDPlayer-Debloat-Block-Begin' }
$rules = (Get-NetFirewallRule -DisplayName 'LDPlayerBlock-*' -ErrorAction
SilentlyContinue).DisplayName
Verify 'firewall egress blocks present'           { ($rules | Measure-Object).Count -ge 1 }

if ($fails -eq 0) { Report 'RESULT: ALL CHECKS PASSED' } else { Report "RESULT: $fails
problem(s) found" }
exit $fails
```

verify-android.ps1

Read-only audit (Android side). Exit code = number of FAILs.

LD · COMMAND

```
# verify-android.ps1 - READ-ONLY Android-side audit over ADB. Run while the emulator is up.
# Prints PASS/FAIL per item. Exit code = number of FAILs.
$LD_ROOT = $null
foreach ($candidate in
@('C:\LDPlayer\LDPlayer9','C:\LDPlayer\LDPlayer12','C:\LDPlayer\LDPlayer5','C:\LDPlayer\LDPlay
er4','C:\LDPlayer\LDPlayer9x')) {
  if (Test-Path "$candidate\adb.exe") { $LD_ROOT = $candidate; break }
}
if (-not $LD_ROOT) {
  $hit = Get-ChildItem 'C:\LDPlayer' -Directory -Force -ErrorAction SilentlyContinue | Where-
Object { Test-Path (Join-Path $_.FullName 'adb.exe') } | Select-Object -First 1
  if ($hit) { $LD_ROOT = $hit.FullName }
}
$adb = "$LD_ROOT\adb.exe"
$fails = 0
function Report($l){ Write-Host $l }
function Verify($name, [scriptblock]$test){
  $ok = $false
  try { $ok = [bool](& $test) } catch { $ok = $false }
  if ($ok) { Report "[PASS] $name" } else { Report "[FAIL] $name"; $script:fails++ }
}

$dev = (& $adb devices | Out-String)
if ($dev -notmatch 'emulator-\d+\s+device') { Report 'SKIP: no emulator attached (start it,
then re-run)'; exit 0 }
Report "ADB: connected ($LD_ROOT)"

$thirdParty = (& $adb shell pm list packages -3 | Out-String)
$disabled   = (& $adb shell pm list packages -d | Out-String)
$dns        = (& $adb shell settings get global private_dns_specifier | Out-String).Trim()
$mode       = (& $adb shell settings get global private_dns_mode | Out-String).Trim()
$home       = (& $adb shell cmd package resolve-activity --brief -a android.intent.action.MAIN
-c android.intent.category.HOME | Out-String).Trim()

Verify 'Lawnchair installed'          { $thirdParty -match 'ch.deletescape.lawnchair' }
Verify 'Aurora Store installed'       { $thirdParty -match 'com.aurora.store' }
Verify 'Magisk installed'             { $thirdParty -match 'com.topjohnwu.magisk' }
Verify 'Private DNS = adguard'        { $dns -eq 'dns.adguard.com' -and $mode -eq 'hostname' }
Verify 'Lawnchair is HOME'            { $home -match 'ch\.deletescape\.lawnchair' }

foreach ($pat in @('launcher3','ld.appstore')) {
  $present = (& $adb shell pm list packages | Out-String) -match [regex]::Escape($pat)
  $dis = $disabled -match [regex]::Escape($pat)
  if ($present) { Verify "LD package ($pat) disabled" { $dis } }
}
$all = (& $adb shell pm list packages | Out-String) -split "`n" | ForEach-Object { $_ -replace
'package:','' } | ForEach-Object { $_.Trim() } | Where-Object { $_ }
$anythingSuspicious = $false
foreach ($p in $all) {
  if ($p -match 'ldmnq|\.ld(-|\.|$)|ldupload|ldperfect') { $anythingSuspicious = $true }
}
Verify 'no obvious LD-promoted packages remain' { -not $anythingSuspicious }

if ($fails -eq 0) { Report 'RESULT: ALL ANDROID CHECKS PASSED' } else { Report "RESULT: $fails
problem(s) found" }
exit $fails
```
